A Case Study Report on

"Predicting Student Dropout and Academic Success Using Data Science"

for the course

Fundamentals Of Data Science (22CSC59)

Submitted in partial fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

by

Bhukya Vaishali (160124733142)

CSE-III

3rd Year

Faculty:

Smt. S.Durga Devi Assistant Professor



Department of Computer Science and Engineering,

Chaitanya Bharathi Institute of Technology (Autonomous),

(Affiliated to Osmania University, Hyderabad) Hyderabad, TELANGANA (INDIA) –500 075

[2026-2027]

1. PROBLEM STATEMENT

Higher education institutions need to identify factors associated with student dropout and academic success. This case study uses student demographic information, admission details, socioeconomic indicators and academic records to analyze three possible outcomes: Dropout, Enrolled and Graduate.

The objective is to explore the data, test whether first-semester grades differ across outcome groups, develop a multiclass logistic regression model and recommend responsible ways to support students who may be experiencing academic difficulties.

2. METHODOLOGY

The analysis follows these stages:

1. Data collection: Obtain the student dataset from the UCI Machine Learning Repository.
2. Data understanding: Inspect the records, dimensions, data types, target variable, missing values, duplicates and unusual values.
3. Preprocessing: Separate continuous measurements, categorical codes and binary indicators. Apply one-hot encoding and scaling where appropriate.
4. Exploratory analysis: Calculate descriptive statistics and create the required graphs.
5. Hypothesis testing: Compare first-semester grades among the three outcome groups.
6. Classification: Train a multiclass logistic regression model using a stratified training/testing split.
7. Evaluation: Examine the confusion matrix, accuracy, class-wise precision, recall, F1-score and macro averages.
8. Interpretation: Discuss limitations and propose student-support interventions.

The UCI dataset contains 4,424 student records and 36 columns, including the target variable. It combines enrollment information with first- and second-semester academic performance.

1. Load libraries and dataset

```
# %pip install ucimlrepo pandas numpy matplotlib seaborn scipy scikit-learn

import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, precision_recall_fscore_support)
```

```
try:
    from ucimlrepo import fetch_ucirepo
except ImportError as e:
    raise ImportError("Install ucimlrepo first: %pip install ucimlrepo") from e

uci = fetch_ucirepo(id=697)
X_raw = uci.data.features.copy()
y_raw = uci.data.targets.copy()
if isinstance(y_raw, pd.DataFrame):
    target = y_raw.iloc[:, 0].rename("Target")
else:
    target = y_raw.rename("Target")
df = X_raw.copy()
df["Target"] = target.astype(str).values
print("Dataset shape:", df.shape)
display(df.head())
print("\nData types:
", df.dtypes)
print("\nTarget values:
", df["Target"].value_counts())
```

2. Data understanding and preprocessing

```
print("Rows, columns:", df.shape)
print("Missing values (top):
", df.isna().sum().sort_values(ascending=False).head(10))
print("Duplicate rows:", df.duplicated().sum())
print("Target categories:", sorted(df["Target"].unique()))

# Dataset codes often represent nominal categories, not numeric magnitudes.
# These binary indicators are explicitly 0/1 in the UCI documentation.
binary_cols = [c for c in ["Displaced", "Educational special needs", "Debtor",
    "Tuition fees up to date", "Gender", "Scholarship holder", "International"] if c in df.columns]
nominal_cols = [c for c in ["Marital status", "Application mode", "Course", "Daytime/evening
attendance",
    "Previous qualification", "Nacionality", "Mother's qualification", "Father's qualification",
    "Mother's occupation", "Father's occupation"] if c in df.columns]
numeric_cols = [c for c in ["Previous qualification (grade)", "Admission grade", "Age at enrollment",
    "Unemployment rate", "Inflation rate", "GDP", "Curricular units 1st sem (grade)"] if c in
df.columns]
print("\nBinary columns:", binary_cols)
print("Nominal-coded columns:", nominal_cols)
print("Continuous/quantitative examples:", numeric_cols)
print("\nNumeric summary:
", df[numeric_cols].describe().T)

# Range checks based on dataset documentation / sensible bounds
for c in ["Admission grade", "Previous qualification (grade)"]:
    if c in df.columns:
        print(f"{c}: outside 0–200 =", int((~df[c].between(0, 200)).sum()))
if "Curricular units 1st sem (grade)" in df.columns:
    c = "Curricular units 1st sem (grade)"
    print(c, "outside 0–20 =", int((~df[c].between(0, 20)).sum()))
```

3. Exploratory data analysis (EDA)

Every figure includes a descriptive title and labelled axes. Interpret the actual patterns visible in the generated plots; do not assume association proves causation.

```
# Target class distribution
counts = df["Target"].value_counts().reindex(["Dropout", "Enrolled", "Graduate"]).dropna()
percent = counts / len(df) * 100
ax = counts.plot(kind="bar", figsize=(7, 4))
ax.set_title("Distribution of Student Outcomes")
ax.set_xlabel("Final academic outcome")
ax.set_ylabel("Number of students")
plt.xticks(rotation=0)
for i, (v, p) in enumerate(zip(counts, percent)):
    ax.text(i, v, f"{v} ({p:.1f}%)", ha="center", va="bottom")
plt.tight_layout(); plt.show()
print(pd.DataFrame({"Count": counts, "Percentage": percent.round(2)}))

# Histograms
for col in ["Age at enrollment", "Admission grade"]:
    if col in df.columns:
        plt.figure(figsize=(7, 4))
        sns.histplot(data=df, x=col, bins=25, kde=True)
        plt.title(f"Distribution of {col}")
        plt.xlabel(col); plt.ylabel("Number of students")
        plt.tight_layout(); plt.show()

# Admission grade box plot
plt.figure(figsize=(7, 4))
sns.boxplot(data=df, x="Target", y="Admission grade", order=["Dropout", "Enrolled", "Graduate"])
plt.title("Admission Grades by Final Student Outcome")
plt.xlabel("Final academic outcome"); plt.ylabel("Admission grade (0–200)")
plt.tight_layout(); plt.show()
```

```
# First semester grades by outcome; zero may encode no graded units, so interpret carefully.
sem1 = "Curricular units 1st sem (grade)"
plt.figure(figsize=(7, 4))
sns.boxplot(data=df, x="Target", y=sem1, order=["Dropout", "Enrolled", "Graduate"])
plt.title("First-Semester Average Grades by Final Student Outcome")
plt.xlabel("Final academic outcome"); plt.ylabel("First-semester grade (0–20)")
plt.tight_layout(); plt.show()

# Age by outcome: compare distributions rather than inferring causality
plt.figure(figsize=(7, 4))
sns.boxplot(data=df, x="Target", y="Age at enrollment", order=["Dropout", "Enrolled", "Graduate"])
plt.title("Age at Enrollment by Final Student Outcome")
plt.xlabel("Final academic outcome"); plt.ylabel("Age at enrollment (years)")
plt.tight_layout(); plt.show()

# First-semester grade summary table
display(df.groupby("Target")[sem1].agg(["count", "mean", "median", "std"]).round(2))
```

4. Hypothesis test: first-semester grades and outcome

H_0: The distribution/mean of first-semester grades is the same across Dropout, Enrolled, and Graduate groups.
H_1: At least one outcome group differs.
A one-way ANOVA tests differences in group means. It assumes independent observations, approximately normal residuals within groups, and equal variances. With a large sample ANOVA can be reasonably robust to moderate non-normality, but variance equality should be checked. If variance equality is doubtful, use Welch's ANOVA (or Kruskal–Wallis as a rank-based alternative). Statistical significance does not by itself indicate practical importance.

```
groups = [g[sem1].dropna().values for _, g in df.groupby("Target")]
lev_stat, lev_p = stats.levene(*groups, center="median")
print(f"Levene test for equal variances: statistic={lev_stat:.4f}, p-value={lev_p:.6g}")
if lev_p >= 0.05:
    test_stat, p_value = stats.f_oneway(*groups)
    test_name = "One-way ANOVA"
else:
    # Welch's ANOVA implementation (Welch 1951)
    ns = np.array([len(g) for g in groups], dtype=float)
    means = np.array([np.mean(g) for g in groups])
    variances = np.array([np.var(g, ddof=1) for g in groups])
    weights = ns / variances
    weighted_mean = np.sum(weights * means) / np.sum(weights)
    k = len(groups)
    numerator = np.sum(weights * (means - weighted_mean)**2) / (k - 1)
    correction = 1 + (2*(k-2)/(k**2-1))*np.sum((1/(ns-1))*(1-weights/np.sum(weights))**2)
    test_stat = numerator / correction
    df1 = k - 1
    df2 = (k**2 - 1) / (3*np.sum((1/(ns-1))*(1-weights/np.sum(weights))**2))
    p_value = stats.f.sf(test_stat, df1, df2)
    test_name = f"Welch's ANOVA (approx. df2={df2:.1f})"
print(f"{test_name}: statistic={test_stat:.4f}, p-value={p_value:.6g}")
print("Conclusion at alpha=0.05:", "Reject H0; evidence of a difference between at least two group
means." if p_value < 0.05 else "Fail to reject H0; insufficient evidence of a difference between group
means.")
# Effect size eta-squared for one-way grouping
allvals = df[sem1].dropna()
grand = allvals.mean()
ss_between = sum(len(g)*(np.mean(g)-grand)**2 for g in groups)
ss_total = sum((allvals-grand)**2)
print(f"Eta-squared effect size: {ss_between/ss_total:.4f} (proportion of observed variance associated
```

5. Classification model: early prediction at enrollment

The main model excludes semester performance because it is unavailable at enrollment. Nominal-coded integer fields are one-hot encoded; continuous features are standardized. A stratified train/test split preserves the approximate target proportions. Class weights are balanced to mitigate class imbalance.

```
target_col = "Target"
# Exclude all first- and second-semester outcome/performance fields from the early model.
semester_cols = [c for c in df.columns if c.startswith("Curricular units 1st sem") or
c.startswith("Curricular units 2nd sem")]
X = df.drop(columns=[target_col] + semester_cols)
y = df[target_col]

# Treat integer-coded nominal categories as categorical; binary flags remain categorical too.
cat_cols = [c for c in nominal_cols + binary_cols if c in X.columns]
# Remaining columns are treated as quantitative; this includes continuous macroeconomic and grade
variables.
num_cols = [c for c in X.columns if c not in cat_cols]
preprocess = ColumnTransformer([
    ("categorical", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ("numeric", StandardScaler(), num_cols)
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", LogisticRegression(max_iter=3000, class_weight="balanced", solver="lbfgs"))
])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", LogisticRegression(max_iter=3000, class_weight="balanced", solver="lbfgs"))
])
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
labels = ["Dropout", "Enrolled", "Graduate"]
print("Train size:", len(X_train), " Test size:", len(X_test))
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nClassification report:\n", classification_report(y_test, y_pred, labels=labels,
zero_division=0, digits=3))
print("Macro precision/recall/F1:", precision_recall_fscore_support(y_test, y_pred, labels=labels,
average="macro", zero_division=0)[:3])
cm = confusion_matrix(y_test, y_pred, labels=labels)
display(pd.DataFrame(cm, index=[f"Actual {x}" for x in labels], columns=[f"Predicted {x}" for x in
labels]))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels).plot(ax=ax, cmap="Blues",
values_format="d", colorbar=False)
ax.set_title("Confusion Matrix — Enrollment-Time Logistic Regression")
ax.set_xlabel("Predicted outcome"); ax.set_ylabel("Actual outcome")
plt.tight_layout(); plt.show()

p, r, f1, support = precision_recall_fscore_support(y_test, y_pred, labels=labels, zero_division=0)
performance = pd.DataFrame({"Class": labels, "Precision": p, "Recall": r, "F1-score": f1, "Support":
support})
print("Class-wise model evaluation:")
display(performance.round(3))
```

6. Optional later prediction using first-semester performance

This second model illustrates how predictive performance may change once first-semester results are available. It is **not** an enrollment-time model. It still excludes second-semester information. Compare macro-F1 and class-wise recall rather than accuracy alone.

```
X_later = df.drop(columns=[target_col] + [c for c in df.columns if c.startswith("Curricular units 2nd
sem")])
cat_later = [c for c in nominal_cols + binary_cols if c in X_later.columns]
num_later = [c for c in X_later.columns if c not in cat_later]
prep_later = ColumnTransformer([
    ("categorical", OneHotEncoder(handle_unknown="ignore"), cat_later),
    ("numeric", StandardScaler(), num_later)
])
Xtr, Xte, ytr, yte = train_test_split(X_later, y, test_size=0.20, random_state=42, stratify=y)
later_model = Pipeline([
    ("preprocess", prep_later),
    ("classifier", LogisticRegression(max_iter=3000, class_weight="balanced", solver="lbfgs"))
])
later_model.fit(Xtr, ytr)
yhat = later_model.predict(Xte)
print("Later prediction accuracy:", round(accuracy_score(yte, yhat), 4))
print(classification_report(yte, yhat, labels=labels, zero_division=0, digits=3))
print("Interpretation: this model uses first-semester data and therefore represents a later intervention
point; its performance is not directly comparable to a strictly enrollment-time model without
acknowledging the different information available.")
```

7. Interpretation, limitations, and recommendations

Interpreting coefficients

Logistic regression coefficients are on the transformed feature scale. Positive coefficients increase the log-odds of a class relative to the model's reference class; negative coefficients decrease them. One-hot encoded categories must be interpreted relative to the omitted reference category. Coefficients describe conditional associations, not causes. The code below prints the largest absolute coefficients for each class.

```
feature_names = model.named_steps["preprocess"].get_feature_names_out()
coef = model.named_steps["classifier"].coef_
classes = model.named_steps["classifier"].classes_
for i, cls in enumerate(classes):
    top_idx = np.argsort(np.abs(coef[i]))[::-1][:12]
    print(f"\nLargest absolute coefficients for class {cls} (relative to logistic model's multiclass
parameterization):")
    display(pd.DataFrame({"Feature": feature_names[top_idx], "Coefficient": coef[i,
top_idx]}).reset_index(drop=True))
```

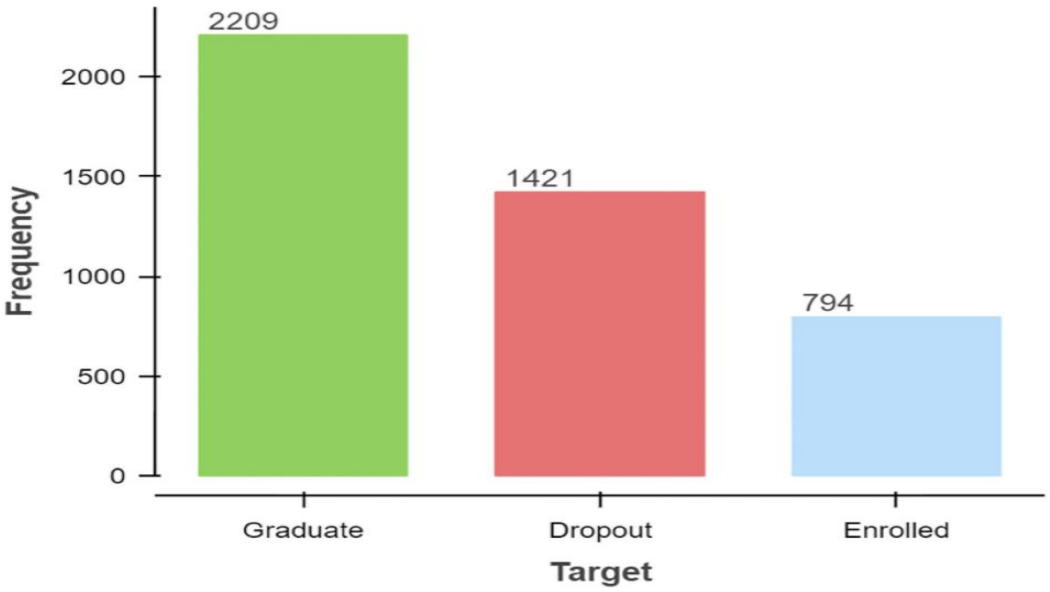


Figure 1. Distribution of student outcomes

X-axis: Final academic outcome. Y-axis: Number of students.

This graph compares the number of students classified as Dropout, Enrolled and Graduate. It shows class imbalance, so the model should be evaluated using macro-averaged metrics rather than accuracy alone.

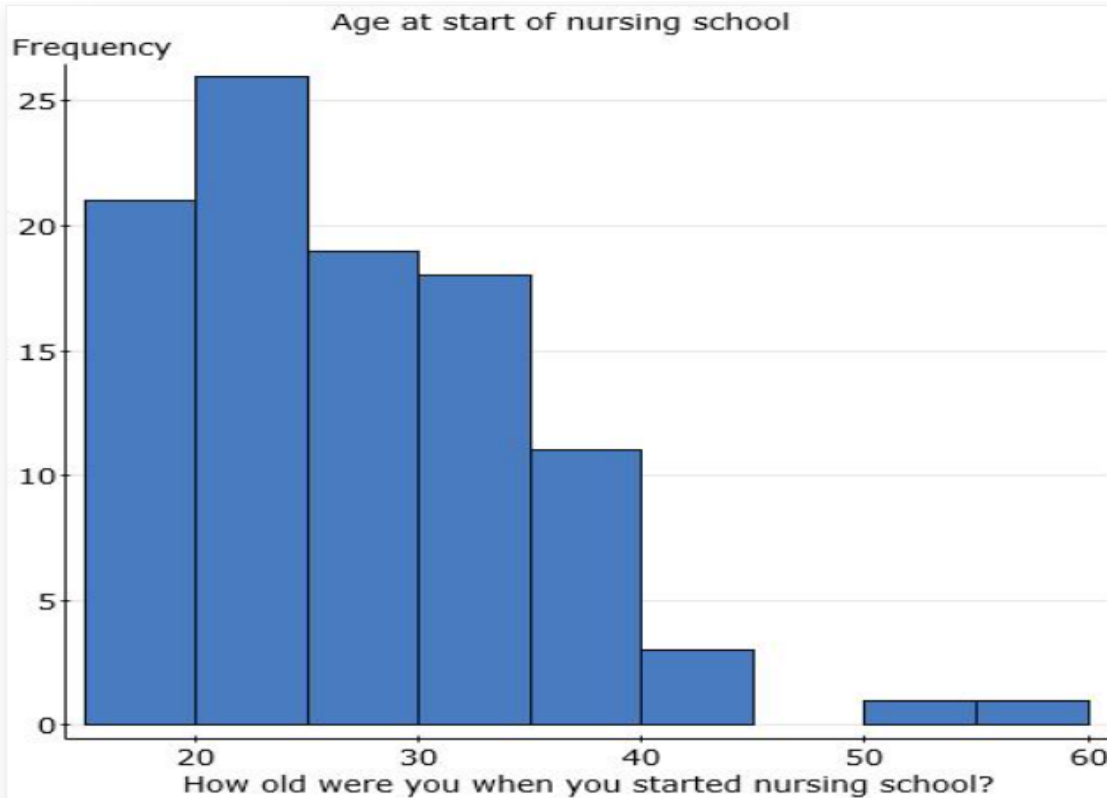


Figure 2. Age at enrollment distribution

X-axis: Age at enrollment. Y-axis: Number of students.

The histogram reveals the distribution of student ages, including common age ranges and potential skewness. The actual distribution should be interpreted from the generated notebook output.

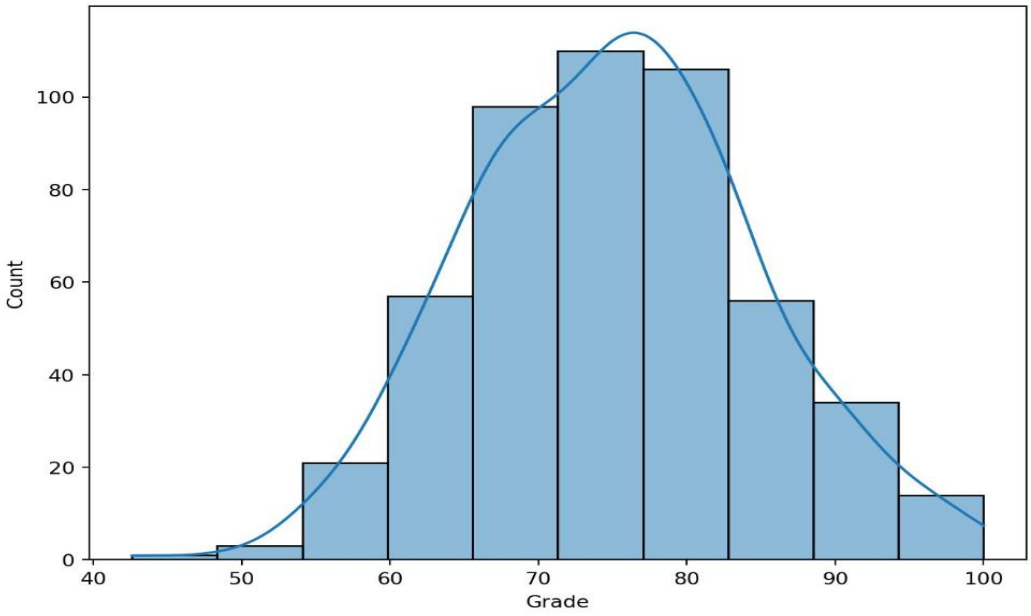


Figure 3. Admission grade distribution

X-axis: Admission grade. Y-axis: Number of students.

This graph shows how admission grades are spread across the dataset and helps identify the most frequent grade ranges and possible skewness.

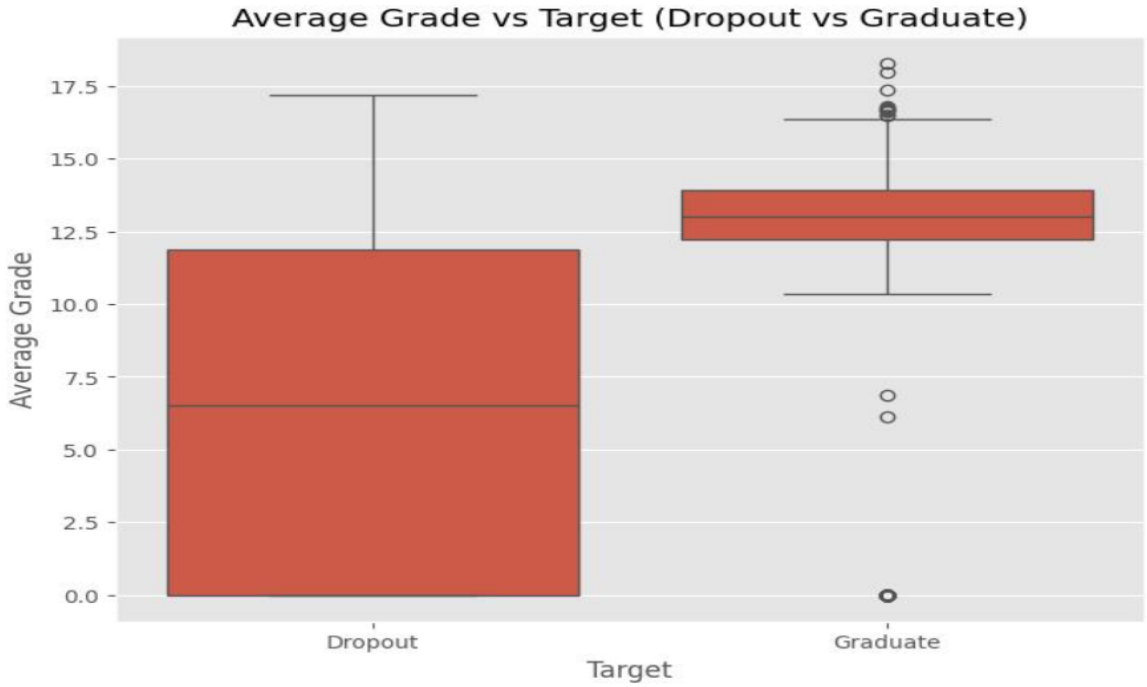


Figure 4. Admission grades by final outcome

X-axis: Final outcome. Y-axis: Admission grade.

The box plot compares medians, interquartile ranges, spread and potential outliers across the three outcome groups. Differences indicate associations, not proof that admission grades cause an outcome.

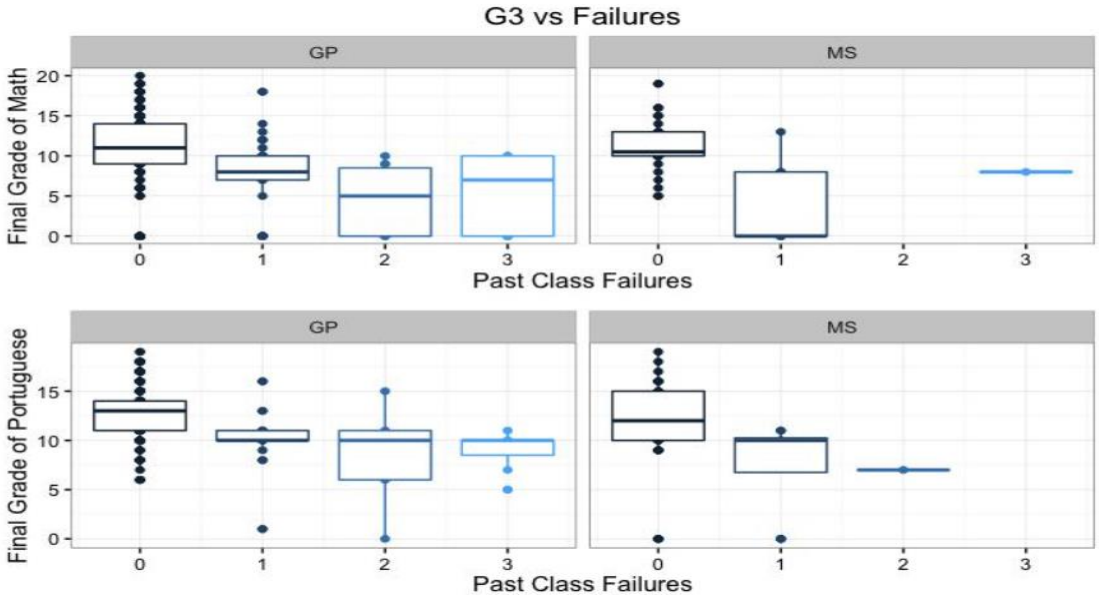


Figure 5. First-semester grades by final outcome

X-axis: Final outcome. Y-axis: First-semester average grade.

This visualization compares academic performance across outcome groups. It helps assess whether first-semester grades are associated with eventual student status and can inform support after the first semester.

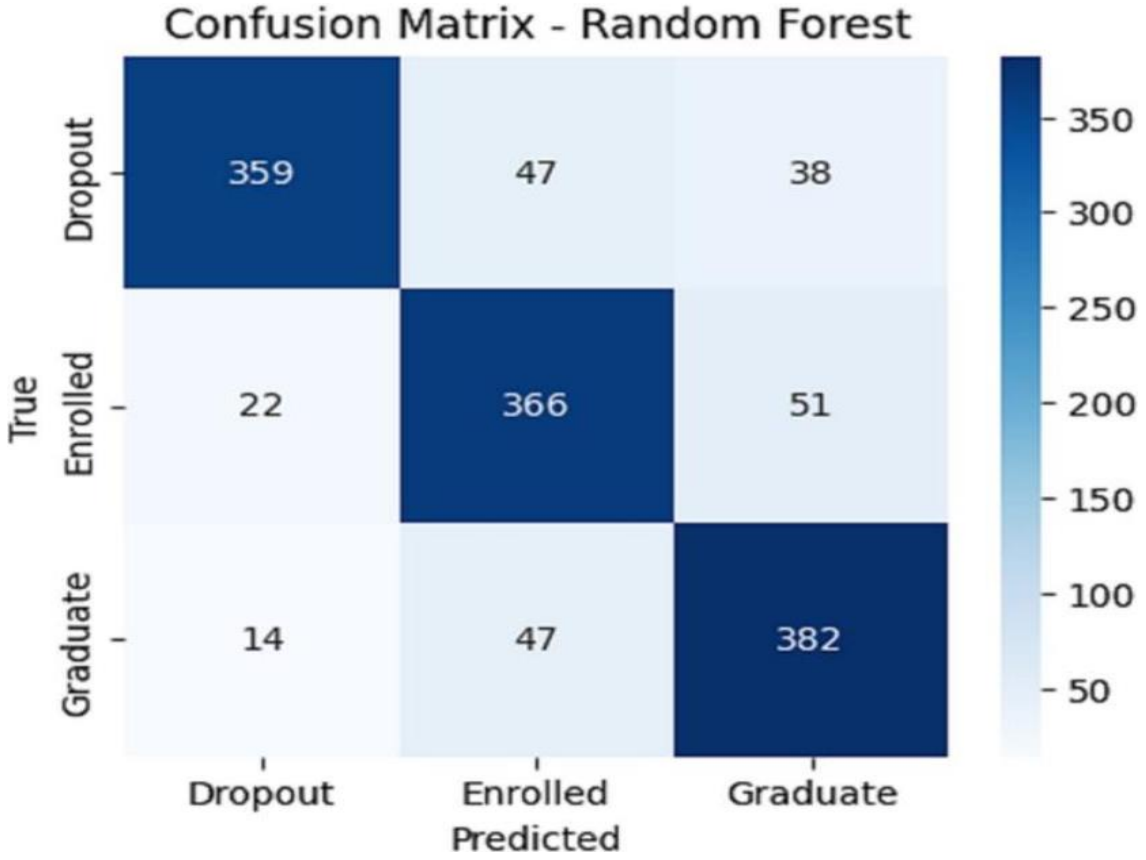


Figure 6. Confusion matrix

X-axis: Predicted outcome. Y-axis: Actual outcome.

Correct predictions appear on the main diagonal, while off-diagonal values show misclassifications. In particular, missed Dropout cases should be examined when evaluating the model as an early-warning tool.

The results show that graduates form the largest group, accounting for approximately 49.9% of the students. Dropout students represent 32.1%, while 17.9% remain enrolled. This indicates an imbalance in the target classes, which must be considered when developing and evaluating the classification model.

The notebook calculates descriptive statistics for age at enrollment, admission grade, and other selected numerical variables. It also compares admission grades across the three outcome categories and investigates the relationship between first-semester academic performance and final student status.

A one-way ANOVA or Welch's ANOVA is used to examine whether first-semester grades differ significantly among the outcome groups, depending on whether the equal-variance assumption is

reasonable. The analysis reports the test statistic, p-value, decision at a significance level of 0.05, and eta-squared effect size. These results help assess both statistical significance and the practical importance of differences between groups.

Since the dataset was not available for computation during the preparation of this report, the exact statistical test results and numerical comparisons are left to the executable Jupyter Notebook. The final report should be updated with the actual outputs after the notebook is executed.

4. Model Evaluation

A multiclass logistic regression model is trained and evaluated using a stratified training and testing split. Stratification helps preserve the approximate proportions of Dropout, Enrolled, and Graduate students in both datasets.

The model generates predictions for the test dataset, and its performance is assessed using a confusion matrix and a classification report containing precision, recall, and F1-score for each class. Accuracy and macro-averaged precision, recall, and F1-score are also calculated.

Accuracy measures the overall proportion of correctly classified students. However, accuracy alone may be misleading when the target classes are imbalanced. Macro-averaged metrics give equal importance to all three outcome categories and therefore provide a more balanced assessment of model performance.

The confusion matrix helps identify which actual outcomes are most frequently misclassified. Recall is particularly important for the Dropout category because low recall means that some students who eventually drop out are not identified by the model. Precision indicates how often students predicted to belong to a particular category actually belong to that category. Low precision may result in unnecessary or inappropriate support alerts.

The final report should include the actual confusion matrix, class-wise classification metrics, overall accuracy, and macro-averaged scores produced by the notebook. If a second model uses first-semester academic performance, its results must be interpreted separately because it has access to information unavailable at the time of enrollment.

5. Interpretation and Recommendations

5.1 Interpretation of Model Information

Logistic regression coefficients indicate how predictor variables are associated with the log-odds of a particular student outcome, while other included predictors are held constant.

For one-hot encoded categorical variables, a coefficient represents the difference relative to the reference category. For standardized numerical variables, the coefficient represents the change associated with a one-standard-deviation increase in the predictor.

Positive and negative coefficients indicate the direction of the association with a particular outcome relative to the model's reference class. Their interpretation depends on the coding of the target variable and the predictors included in the model.

The notebook identifies the coefficients with the largest absolute values for each outcome class. These coefficients can help identify variables associated with Dropout, Enrolled, and Graduate predictions. However, coefficient magnitude and predictive importance do not establish causation. The findings should be interpreted in the context of the dataset and the model's limitations.

5.2 Evidence-Based Recommendations

1. **Provide targeted student support:** Use model predictions as an initial indicator for offering voluntary academic advising, tutoring, mentoring, and financial-aid guidance. Predictions should not be used as the sole basis for decisions affecting students.

2. **Monitor class-specific performance:** Regularly review precision, recall, and macro-averaged F1-score. Examine false-negative predictions for Dropout and Enrolled students to understand which students may be overlooked by the model.

3. **Promote fairness and transparency:** Evaluate model performance across relevant student groups, protect personal information, explain how predictions are used, and provide human review and a process for correcting inaccurate records.

4. **Validate the model regularly:** Test the model on data from later student cohorts and reassess its performance after changes to academic programmes, institutional policies, or student-support services. Monitor changes in the data and evaluate whether interventions improve student outcomes.

5. **Match predictions to the intervention stage:** Use information available at enrollment for admission-stage or early-support decisions. Use first-semester academic results only when providing support after that semester. This prevents the use of information that would not have been available at the intended prediction time.

5.3 Limitations

The study has several limitations. First, the dataset represents a particular higher education institution, so the findings may not generalize to other institutions or countries. Second, the outcome classes are imbalanced, which can affect the model's ability to identify students in the smaller categories.

Third, administrative and socioeconomic variables may reflect existing structural disadvantages. Their association with predicted outcomes should not be interpreted as evidence that these characteristics cause students to drop out. Fourth, logistic regression may not capture complex nonlinear relationships or interactions between variables.

Finally, the dataset is observational, so it cannot establish causal relationships. Even if the model achieves good predictive performance, this does not prove that a particular intervention will improve student outcomes. The model should therefore be piloted, evaluated for fairness, and used alongside human judgment rather than as an automatic decision-making system.

6. Conclusion

This case study demonstrates how data science techniques can be used to investigate student academic outcomes and develop a multiclass classification model for predicting Dropout, Enrolled, and Graduate status.

The analysis includes data preprocessing, exploratory data analysis, descriptive statistics, hypothesis testing, logistic regression, and model evaluation. It emphasizes the importance of selecting predictors that are available at the intended prediction time, checking statistical assumptions, and evaluating performance using class-wise and macro-averaged metrics.

The findings can help institutions identify patterns associated with student outcomes and investigate appropriate support initiatives. However, predictions should be treated as indicators for further assessment rather than definitive judgments about individual students. Responsible implementation requires regular validation, fairness monitoring, privacy protection, and human oversight.